University of Tunis El Manar

National Engineering School of Tunis

Information and Communication Technologies Department

Rapport de projet de fin d'année II

Mise en place d'un simulateur basé sur l'IA pour la détection d'intrusions dans les applications IoT

Rédigé par:

Fares Insaf

Class:

2ème Année TEL

Supervisé par

Mr. Kouki Tarek

Academic Year: 2025-2026

Dédicaces

Je dédie ce modeste travail :

À mes chers parents, pour leur soutien et leurs prières,

À mes chères sœurs,

À tous mes amis,

À tous ceux qui me sont chers.

Remerciements

Avant tout, je tiens à adresser un grand merci à Allah, qui m'a accordé la santé, la volonté, le courage et la détermination nécessaires pour mener à bien ce travail. e souhaite également exprimer ma profonde gratitude à Monsieur Tarek Kouki, mon encadrant de PFA, pour son aide précieuse, son soutien constant et la qualité de ses conseils tout au long de ce projet.

Je remercie également les membres du jury, qui ont bien voulu consacrer de leur temps pour assister à ma soutenance et évaluer mon travail.

Enfin, à toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce projet, je dis merci du fond du cœur.

Merci à toutes et à tous.

Abstract

With the widespread adoption of the Internet of Things (IoT), smart environments are increasingly integrated into various domains such as home automation, healthcare, industry, and smart cities. However, this expansion brings growing security challenges, as IoT systems are often resource-constrained and vulnerable to a wide range of attacks. To address this issue, this work proposes the design and implementation of an AI-based simulator for intrusion detection in IoT applications. The project is built around a comprehensive study of traditional and modern detection techniques, with a focus on machine learning and deep learning approaches. Special attention is given to the development of a hybrid CNN-LSTM detection model, which is evaluated through experimental simulations. The results highlight the effectiveness of AI-based techniques in enhancing IoT network security. This simulator offers a flexible platform for experimenting with, comparing, and improving different intrusion detection strategies tailored to the specific constraints of IoT architectures.

Keywords : Internet of Things, Artificial Intelligence, Intrusion Detection, Deep Learning, Simulation, Cybersecurity

Résumé

Avec la généralisation de l'Internet des Objets (IoT), les environnements intelligents deviennent de plus en plus répandus dans divers domaines, tels que la domotique, la santé, l'industrie et les villes intelligentes. Toutefois, cette expansion s'accompagne d'une multiplication des menaces de sécurité, dues aux vulnérabilités inhérentes aux systèmes IoT, souvent limités en ressources et exposés à divers types d'attaques. Face à cette problématique, notre travail propose la conception et la mise en œuvre d'un simulateur basé sur l'intelligence artificielle (IA) destiné à la détection d'intrusions dans les applications IoT. Ce projet s'articule autour de l'étude des approches de détection classiques et modernes, notamment celles reposant sur le machine learning et le deep learning. Une attention particulière est accordée à la modélisation d'un système de détection hybride CNN-LSTM, évalué à travers des expérimentations en environnement simulé. Les résultats obtenus démontrent la pertinence des techniques basées sur l'IA pour renforcer la sécurité des réseaux IoT. Ce simulateur constitue une base d'expérimentation évolutive pour tester, comparer et affiner différentes stratégies de détection adaptées aux contraintes spécifiques des architectures IoT.

Mots clés : , Intelligence Artificielle, Internet des Objets, Détection d'intrusion, Deep Learning, Simulation, Sécurité informatique

Contents

List of Figures **7**

List of Tables **8**

List of Acronyms **9**

General introduction **10**

1 État de l'Art **12**
Introduction . . . 12
1.1 L'Internet des Objets (IoT) . . . 12
1.1.1 Définition de l'IoT et priorités . . . 13
1.1.2 Architecture d'un système IoT . . . 13
1.1.3 Domaines d'application . . . 14
1.2 Sécurité dans l'IoT . . . 17
1.2.1 Vulnérabilités dans les applications IoT . . . 17
1.2.2 Types d'attaques . . . 18
1.3 Fonctions principales d'un IDS . . . 20
1.3.1 Techniques de détection d'intrusion . . . 20
Conclusion . . . 23

2 Détection d'intrusion basée sur l'intelligence artificielle **24**
2.1 Introduction . . . 24
Introduction . . . 24
2.2 Détection basée sur l'intelligence artificielle . . . 24
2.2.1 Machine Learning . . . 25
2.2.2 Détection d'intrusion basée sur le deep learning . . . 27
2.3 Analyse des Méthodes de Détection d'Attaques . . . 32
Conclusion . . . 33

3 Méthodologie et Conception **35**
Introduction . . . 35
3.1 **Objectifs et Démarche générale** . . . 35
3.1.1 Objectifs de la méthodologie . . . 35
3.1.2 Démarche générale adoptée . . . 36
3.1.3 Méthodologie proposée . . . 36
3.1.4 Préparation des données . . . 37
3.1.5 Choix de l'algorithme d'apprentissage . . . 40
3.1.6 Métriques et évaluation . . . 42
Conclusion . . . 43

4 Implementations et Résulats **44**
Introduction . . . 44
4.1 Environnement d'exécution et Configuration du système . . . 44
4.1.1 Environement d'execution . . . 44
4.1.2 Configuration du système . . . 45
4.2 Résultats expérimentaux . . . 45
4.2.1 Résultats du modèle CNN-LSTM . . . 45
4.3 Implémentation du Système . . . 46
4.3.1 Mise en place de l'environnement de simulation . . . 48
4.3.2 Résultats et observations . . . 51
Conclusion . . . 52

Conclusion **53**
.1 Chargement et Prétraitement des Données . . . 56
.2 Définition de l'Architecture CNN-B . . . 56
.3 Entraînement du Modèle avec Early Stopping . . . 57

Bibliography **58**

List of Figures

1.1 Composition d'un systèmes IoT 13
1.2 architecture d'un systèmes IoT 14
1.3 domaine d'application 17
1.4 Approche par connaissance 20
1.5 Approche comportementale 21

2.1 Domaines de l'intelligence artificielle 25
2.2 Enter Caption 26
2.3 L'architecutre d'un modle de deep learning 27
2.4 architecture de modele DNN 29
2.5 structure d'un CNN 30
2.6 architecture de modele RNN 31
2.7 cellule LSTM 32

3.1 distrubition du dataset 38
3.2 Approche utilisée pour détecter les attaques 42

4.1 architecture de modele CNN-LSTM 45
4.2 Accuracy vs appoch 47
4.3 Loss vs Epoch 47
4.4 Confusion Matrix of the test/predicted attacks in Bot-iot using CNN 47
4.5 accracy,recall 48
4.6 execution de fichier 5g-.cc 48
4.7 Analyse du Premier Paquet avec Wireshark 50
4.8 simulation sur NetAnim 50
4.9 DDOS simulation 51

List of Tables

1.1 Principales méthodes de sécurisation des systèmes IoT 19
1.2 Analyse comparative des IDS basés sur la signature dans les environnements IoT 21
1.3 Analyse comparative de la détection comportementale dans les environnements IoT 22

2.1 Comparaison détaillée entre Machine Learning et Deep Learning . . 28
2.2 Avantages et Inconvénients des RNN 31
2.3 Enquête sur les Attaques et les Approches de Détection Appliquées par les Chercheurs 34

3.1 Répartition des types de trafic dans le sous-ensemble BoT-IoT (10 %) 39
3.2 Description des caractéristiques sélectionnées 40

4.1 Rapport de classification 46

Liste des acronymes

- **IOT:** Internet des Objets
- **IDS:** Systèmes de Détection d'Intrusions
- **ML:** Machine Learning
- **DL** : Deep Learning
- **LSTM:** Long Short-Term Memory.
- **CSV:** Comma-Separated Values (fichier de données tabulées)
- **DNN:** Deep Neural Network (Réseau de Neurones Profond)
- **CNN :** Convolutional Neural Network (Réseau de Neurones Convolutif)
- **RNN:** Recurrent Neural Network.
- **ML:** Machine Learning
- **DoS :** Denial of Service (attaque par déni de service)
- **DDOS :** Tensor Processing Unit.

Introduction générale

Contexte général du sujet

L'Internet des Objets (IoT) représente une avancée significative dans le domaine des technologies modernes, reliant des milliards d'appareils pour la collecte et l'échange de données. Cette interconnexion, bien que bénéfique pour une multitude d'applications, entraîne des défis en matière de sécurité, étant donné la complexité et la variabilité des réseaux IoT. La nécessité de garantir la disponibilité, l'intégrité et la confidentialité des dispositifs connectés est devenue essentielle à mesure que les cybermenaces évoluent et se diversifient.

Contexte spécifique

La sécurité des réseaux IoT est particulièrement préoccupante, compte tenu des vulnérabilités inhérentes à ces systèmes et des différents types d'attaques potentiellement exploitables, telles que les intrusions et les comportements malveillants. Traditionnellement, les systèmes de détection d'intrusions (IDS) utilisés dans ces environnements ont révélé leur limites face à l'avènement de techniques sophistiquées. Il est donc crucial d'explorer des approches innovantes basées sur l'intelligence artificielle pour améliorer la détection et la réaction face à ces menaces.

Description de la solution proposée et des réalisations

Ce rapport propose une méthode intégrant l'intelligence artificielle, spécifiquement le deep learning, pour développer un modèle intelligent de détection d'intrusions dans les réseaux IoT. En combinant une simulation d'environnements IoT avec des modèles d'apprentissage profond, cette approche vise à identifier et à classifier les comportements normaux et anormaux, renforçant ainsi la sécurité et la réactivité des systèmes.

Aperçu du contenu du rapport

Ce rapport est structuré en quatre chapitres, répartis de la manière suivante :

- **Le premier chapitre :**

  Ce chapitre est dédié à l'Internet des Objets (IoT), aux attaques potentielles dans ces environnements et aux systèmes de détection d'intrusions classiques (IDS). Nous y présentons les caractéristiques de l'IoT, les défis de sécurité auxquels il est confronté, ainsi que les types d'attaques possibles. De plus, nous analysons les IDS traditionnels et les méthodes utilisées pour détecter les comportements malveillants dans les systèmes IoT.

- **Le deuxième chapitre :**

  Ce chapitre se concentre sur l'application du Deep Learning (DL) pour améliorer la détection d'intrusions dans l'IoT. Nous explorons les concepts fondamentaux du Deep Learning, comme les réseaux de neurones profonds, et leur application dans la cybersécurité. Nous mettons en évidence comment ces techniques peuvent être utilisées pour détecter des comportements malveillants de manière plus efficace dans les environnements IoT.

- **Le troisième chapitre :**

  Ce chapitre est dédié à la conception et à la méthodologie de la solution proposée pour la détection d'intrusions dans les systèmes IoT. Nous décrivons l'architecture générale du système, les choix technologiques effectués, ainsi que la méthode utilisée pour le développement et la mise en œuvre de la solution.

- **Le dernier chapitre :**

  Ce chapitre présente l'implémentation du système développé ainsi que les résultats obtenus à partir de la simulation et de l'évaluation de notre modèle de détection d'intrusions. Nous détaillons la méthodologie de test, les outils utilisés et les scénarios d'évaluation, puis nous discutons de la performance de notre solution en termes de détection des attaques et de sécurité renforcée.

Chapter 1

État de l'Art

Introduction

L'Internet des Objets (IoT) s'est imposé comme un pilier des technologies modernes, en connectant des milliards d'appareils pour collecter et échanger des données. Cependant, cette interconnexion massive s'accompagne de défis majeurs, notamment en matière de sécurité et de protection des données.

Dans ce chapitre, nous présentons une synthèse des concepts clés liés à l'IoT, aux problématiques de sécurité, ainsi qu'aux techniques classiques de détection d'intrusions. Notre objectif est d'établir une base théorique solide pour comprendre les enjeux du projet et orienter notre démarche de recherche.

Nous débutons par une introduction à l'IoT, en définissant ses caractéristiques principales, son architecture typique et ses domaines d'application. Ensuite, nous analysons les enjeux de sécurité en identifiant les vulnérabilités propres aux environnements IoT et les types d'attaques qui en découlent. Enfin, nous étudions les approches classiques de détection d'intrusions, en exposant les méthodes traditionnellement utilisées pour identifier les comportements malveillants dans les réseaux.

Cette étude préliminaire nous permettra d'appréhender les défis spécifiques liés à la sécurité de l'IoT et de mieux situer notre contribution au sein de cet écosystème.

.

1.1 L'Internet des Objets (IoT)

L'Internet des Objets (IoT) représente une évolution majeure des technologies numériques, visant à connecter des objets physiques à Internet afin de faciliter l'échange de données et d'informations. Grâce à cette interconnexion, de nouvelles

possibilités émergent dans des domaines variés tels que la santé, l'industrie, la domotique ou les transports intelligents.

1.1.1 Définition de l'IoT et priorités

L'IoT constitue un écosystème numérique intelligent, dans lequel des objets physiques sont connectés à Internet pour échanger des données via des protocoles standardisés. Cette connectivité globale permet un accès instantané aux ressources et aux informations, indépendamment du lieu et du moment. Un système IoT repose généralement sur trois composants principaux.

illustré dans la figure 1.1 [1]

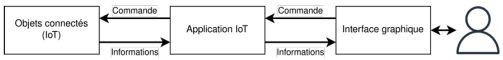


Figure 1.1: Composition d'un systèmes IoT

- un ensemble d'objets intelligents capables de collecter, traiter et transmettre des données.
- une infrastructure réseau permettant la communication entre les objets et les applications.
- Des plateformes logicielles chargées d'analyser les données collectées pour fournir des services à valeur ajoutée.

Les priorités dans le développement de l'IoT sont centrées sur l'amélioration de l'efficacité des processus, l'optimisation de la prise de décision grâce à l'analyse de données massives, et la création d'environnements plus intelligents et adaptatifs.

1.1.2 Architecture d'un système IoT

L'architecture IoT consiste en un ensemble structuré de systèmes interconnectés permettant d'assurer la collecte, le stockage et le traitement des données générées par les capteurs dans des entrepôts de données massifs. De plus, elle garantit que les actionneurs des objets exécutent les commandes transmises via une application utilisateur.

Cette architecture définit les composants physiques, l'organisation fonctionnelle, la configuration du réseau, les procédures opérationnelles et les formats de données à adopter.

Cependant, il convient de noter qu'il n'existe pas de modèle d'architecture standardisé unique pour l'IoT, étant donné la diversité des technologies impliquées.

Éléments de base d'une architecture IoT

L'architecture IoT peut varier considérablement selon l'implémentation, mais elle doit être suffisamment flexible pour prendre en charge diverses applications réseau grâce à l'utilisation de protocoles ouverts.

Généralement, elle repose sur quatre éléments clés :

- **La scalabilité** : capacité d'adaptation à l'augmentation du nombre de dispositifs connectés.
- **La fonctionnalité** : aptitude à fournir les services attendus efficacement.
- **La disponibilité** : garantie d'accès continu aux services et aux données.
- **La maintenabilité** : facilité de mise à jour, de réparation et d'évolution du système.

Bien qu'il n'existe pas d'architecture IoT universellement acceptée, le modèle à quatre couches est le plus simple et le plus largement adopté.[2]

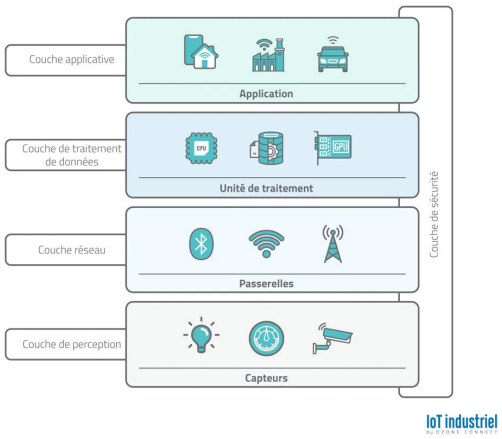


Figure 1.2: architecture d'un systèmes IoT

1.1.3 Domaines d'application

Il y a encore quelques années, les objets connectés étaient peu connus du grand public. Aujourd'hui, ils sont omniprésents dans notre quotidien. Des téléviseurs intelligents aux voitures connectées, ces technologies ont considérablement amélioré notre confort, nos loisirs et nos déplacements.

Le potentiel d'interconnexion des objets est immense. Une étude de Ericsson montre que le nombre pourrait atteindre **environ 50 milliards** d'ici 2030, plus de

la moitié des outils et processus commerciaux seraient accessibles via l'IoT. Les domaines d'application sont nombreux et couvrent un large éventail de secteurs : industrie, recherche scientifique, santé, domotique, agriculture, transport, etc.

De nombreux exemples concrets montrent comment l'IoT transforme notre manière de vivre, de produire et de communiquer. Voici quelques domaines clés d'application :

1. Domotique et Smart Homes

La domotique utilise les IoT pour automatiser et contrôler les appareils domestiques. Les applications incluent :

- **Systèmes de sécurité :** Caméras de surveillance intelligentes, détecteurs de mouvement et alarmes connectées renforcent la sécurité des domiciles.
- **Gestion de l'énergie :** Thermostats intelligents et ampoules connectées permettent de surveiller et d'optimiser la consommation d'énergie.
- **Appareils électroménagers :** Réfrigérateurs, lave-linge et autres appareils peuvent être contrôlés à distance via des applications mobiles, améliorant ainsi la commodité.

2. Santé Connectée

Les solutions IoT en santé offrent une surveillance continue des patients, améliorant la qualité des soins. Les exemples incluent :

- **Dispositifs portables :** Montres intelligentes et capteurs de fitness suivent la santé et l'activité physique des utilisateurs.
- **Surveillance à distance :** Dispositifs connectés tels que les glucomètres et les tensiomètres permettent un suivi à distance, facilitant une intervention rapide en cas de besoin.
- **Gestion des médicaments :** Les systèmes IoT peuvent rappeler aux patients de prendre leurs médicaments à temps, réduisant ainsi les problèmes d'adhérence aux traitements.

3. Transport et Logistique

Les IoT révolutionnent le secteur des transports avec une gestion plus efficace des ressources. Les applications incluent :

- **Suivi des véhicules :** Les systèmes GPS connectés permettent de suivre en temps réel l'emplacement des véhicules, optimisant les itinéraires et réduisant les coûts.
- **Gestion de la chaîne d'approvisionnement :** Les capteurs IoT surveillent les conditions des marchandises (température, humidité) pendant le transport, assurant leur intégrité.
- **Véhicules connectés :** Les voitures intelligentes sont désormais équipées de technologies de communication qui améliorent la sécurité routière et l'expérience de conduite.

4. Agriculture de Précision

Les IoT jouent un rôle crucial dans l'amélioration de l'efficacité agricole. Les exemples incluent :

- **Capteurs de sol :** Mesurent l'humidité et les nutriments, permettant un arrosage et une fertilisation optimisés.
- **Drones agricoles :** Utilisés pour surveiller les cultures, ils fournissent des données précieuses sur la santé des plantes et la répartition des ressources.
- **Systèmes d'irrigation intelligents :** Les systèmes IoT automatisés ajustent les horaires d'irrigation en fonction des conditions climatiques et des besoins des cultures.

5. Villes Intelligentes

Les IoT sont au cœur des initiatives de villes intelligentes, visant à améliorer la qualité de vie urbaine. Les applications comprennent :

- **Gestion de la circulation :** Les capteurs IoT analysent le flux de trafic en temps réel pour ajuster les feux de circulation et réduire les embouteillages.
- **Éclairage public intelligent :** Les lampadaires ajustent leur intensité en fonction de la lumière ambiante et du passage des piétons.
- **Eau et déchets :** Les capteurs surveillent la consommation d'eau et le niveau des déchets, permettant une gestion proactive des ressources urbaines.

Dans les sections suivantes, nous aborderons les défis spécifiques liés à la sécurité des systèmes IoT, ainsi que les approches classiques de détection d'intrusions dans ces environnements.

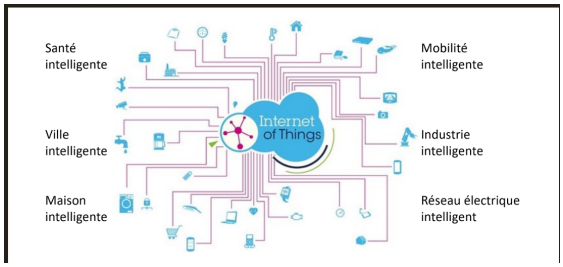


Figure 1.3: domaine d'application

1.2 Sécurité dans l'IoT

Alors que nous récoltons les bénéfices de l'Internet des objets, il est essentiel de ne pas

Ignorer les problèmes de sécurité. En tant que concepteur et utilisateur

Avec l'IoT, nous devons intégrer la sécurité dès la conception. Cela implique

Assurer la sécurité de nos données personnelles et notre sécurité personnelle.

Assurer la sécurité des appareils, des réseaux et des données transitant

L'Internet des objets nécessite la mise en œuvre d'un nouveau modèle de sécurité qui

Assurer l'authenticité et la confidentialité des informations est de la plus haute importance.

1.2.1 Vulnérabilités dans les applications IoT

Une fois connectés à Internet, les objets intelligents deviennent exposés à de nombreux risques. Cette immense infrastructure interconnectée présente plusieurs failles exploitables par des attaquants. Parmi les principales vulnérabilités identifiées, on retrouve :

- **Manque de protection physique** : De nombreux dispositifs sont déployés sans mesures matérielles de protection, les exposant aux manipulations ou sabotages physiques.
-
- **Stockage et transmission de données non sécurisés** : Sans protocoles de chiffrement et contrôle d'accès robustes, le risque de fuite de données augmente.
- **Exploitation par des botnets** : Les objets mal sécurisés sont souvent détournés pour participer à des attaques massives coordonnées.

-
- **Faible flexibilité des applications IoT** : Les conceptions monolithiques limitent l'adaptabilité face aux nouvelles menaces.

1.2.2 Types d'attaques

La diversité des dispositifs IoT ouvre la voie à plusieurs attaques. Ces attaques exploitent les vulnérabilités précédemment citées. On peut distinguer trois grandes catégories d'attaques selon leur cible : le matériel (hardware), les applications et les protocoles de communication.

Attaques sur le Hardware

Les attaques ciblant le matériel visent à exploiter des faiblesses physiques ou techniques des dispositifs IoT :

Attaques par accès physique : Les attaquants peuvent manipuler directement le matériel pour extraire des données ou compromettre son intégrité. Par exemple, la connexion à des ports de débogage (JTAG, UART) peut permettre une modification non autorisée du firmware.

Attaques matérielles :
Ces attaques peuvent comprendre l'utilisation de techniques telles que le side-channel attack, où les attaquants observent des canaux d'information (comme la consommation d'énergie ou les émissions électromagnétiques) pour déduire des informations sensibles telles que des clés de chiffrement.

Piratage physique : L'intrusion physique dans des installations pour voler des appareils IoT ou exploiter des points faibles de sécurité physique, rendant ainsi le système vulnérable à d'autres attaques.

sur les Applications

Les attaques sur les applications ciblent souvent la couche logicielle des dispositifs IoT, en cherchant à exploiter des failles dans les interfaces :

Injection SQL : Les interfaces web des applications IoT peuvent être vulnérables à des attaques d'injection SQL, permettant aux attaquants de manipuler la base de données sous-jacente.

Cross-Site Scripting (XSS) : Les attaquants peuvent insérer des scripts malveillants dans des pages web pour prendre le contrôle de sessions utilisateurs ou voler des informations personnelles.

Exploitation de l'absence de mises à jour de sécurité: Les vulnérabilités dans les protocoles de communication peuvent entraîner des compromissions significatives de la sécurité des données :

Attaques sur les Protocoles de Communication

Les vulnérabilités dans les protocoles de communication peuvent entraîner des compromissions significatives de la sécurité des données :

- **Man-in-the-Middle (MitM) :** Des attaquants interceptent les communications entre un dispositif IoT et un serveur, permettant d'écouter ou de modifier les données échangées.
- **DDoS (Distributed Denial of Service) :** Cette attaque vise à inonder un serveur IoT avec des requêtes afin de le rendre hors service, par une hostilité coordonnée utilisant de nombreux dispositifs compromis.
- **Interceptions de données non chiffrées :** Le manque de chiffrement dans les communications expose les données sensibles à des interceptions faciles, compromettant la confidentialité et l'intégrité des informations transmises.

Synthèse et choix d'approche

Face à la diversité des vulnérabilités et des types d'attaques identifiés dans les systèmes IoT, plusieurs méthodes de sécurisation ont été proposées, comme le montre le tableau 1.1. Ces méthodes couvrent la protection des données, l'authentification, la sécurité réseau et la surveillance du comportement des dispositifs.

Table 1.1: Principales méthodes de sécurisation des systèmes IoT

Catégorie	**Méthode de sécurisation**	**Description**
1	Partage et sécurité des données	Chiffrement, anonymisation, contrôles d'intégrité des flux IoT
2	Contrôle d'accès	Politique d'autorisation pour restreindre l'accès aux ressources
3	Authentification	Vérification d'identité des objets et des utilisateurs
4	Sécurité réseau	Détection d'intrusion, segmentation et firewalls
5	Modèles et méthodes de sécurité	Conception formelle de protocoles sûrs et vérification
6	Résilience et maintenabilité	Tolérance aux pannes, continuité de service et mises à jour
7	Disponibilité	Protection contre DDoS, jamming, maintien de la QoS
8	Surveillance de sécurité	Analyse comportementale, détection d'anomalies

Toutefois, dans un environnement IoT où les attaques peuvent être rapides, complexes et parfois inconnues, la capacité à détecter les anomalies devient primordiale pour assurer une protection efficace.

C'est pourquoi notre travail s'est focalisé sur la détection d'anomalies, afin de renforcer la résilience globale des applications IoT face aux menaces actuelles.

1.3 Fonctions principales d'un IDS

Le premier modèle de détection d'intrusion a été élaboré en 1984 par Dorothy Denning, en collaboration avec Peter Neuman. Fondé sur une approche comportementale à base de règles, ce modèle a donné naissance à l'Intrusion Detection Expert System (IDES), développé pour la première fois en 1988. Considéré comme l'un des précurseurs des systèmes de détection d'intrusion (IDS), IDES avait pour mission de surveiller le trafic réseau, entrant et sortant, afin d'identifier toute activité anormale, suspecte ou malveillante. En cas de détection d'une menace potentielle, une alerte était transmise aux responsables informatiques pour qu'ils puissent analyser la situation et prendre les mesures appropriées pour contrer l'attaque.

1.3.1 Techniques de détection d'intrusion

Détection par signature

: Les systèmes classiques de détection d'intrusion (IDS) fonctionnent principalement selon un modèle de détection par signature. Ce mécanisme repose sur l'analyse statique des données échangées dans le réseau, en les comparant à une base de signatures d'attaques préalablement connues. Lorsqu'un modèle ou un motif correspondant à une attaque enregistrée est détecté, le système génère une alerte.[3] Dans les en-

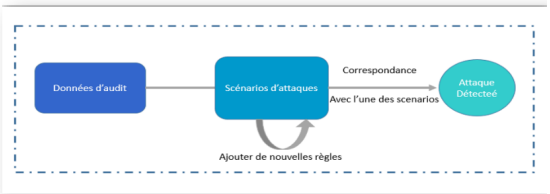


Figure 1.4: Approche par connaissance

vironnements IoT, les IDS basés sur la signature présentent à la fois des avantages et des limites, comme détaillé dans le tableau ci-dessous:

Table 1.2: Analyse comparative des IDS basés sur la signature dans les environnements IoT

Avantages	Limitations
Faible consommation de ressources : adaptée aux contraintes matérielles des dispositifs IoT **Rapidité de détection :** identification quasi-instantanée des menaces connues **Facilité de mise en œuvre :** ajout simple de nouvelles signatures	**Incapacité à détecter les attaques inconnues :** vulnérabilité aux attaques zero-day ou modifiées **Maintenance lourde :** mise à jour constante nécessaire de la base de signatures **Adaptabilité réduite :** difficulté à suivre le comportement dynamique des objets connectés

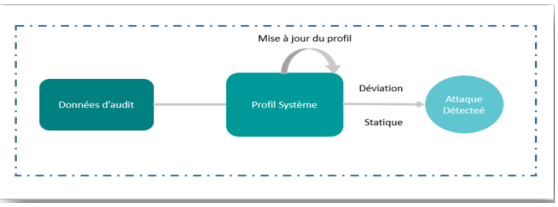


Figure 1.5: Approche comportementale

Exemples typiques d'IDS basés sur la signature utilisés ou adaptés pour les réseaux IoT :Snort Suricata Bro/Zeek (modifié pour l'IoT)

Ainsi, même si les IDS basés sur la signature restent très utilisés pour la sécurité des systèmes IoT, ils sont souvent complétés par des méthodes comportementales ou hybrides pour pallier leurs faiblesses.

Détection comportementale pour les applications IoT

La détection comportementale, également appelée **anomaly-based detection**, repose sur l'analyse des **écarts** entre le comportement actuel du système et un **profil de référence** établi en période normale. Plutôt que de chercher une correspondance avec des signatures connues, cette méthode identifie des comportements **inhabituels** qui pourraient signaler une attaque.[4] Dans les environnements IoT, les IDS basés sur la Détection comportementale présentent à la fois des avantages et des limites, comme détaillé dans le tableau ci-dessous:

Exemples de systèmes ou de projets qui utilisent une détection comportementale adaptée à l'IoT

Table 1.3: Analyse comparative de la détection comportementale dans les environnements IoT

Avantages	Défis Spécifiques
Capacité à détecter des attaques inconnues : identification des attaques zero-day et des variations subtiles **Adaptabilité :** ajustement des profils de normalité selon l'évolution du réseau **Détection proactive :** identification précoce des activités anormales	**Taux de faux positifs élevé :** risque de mauvaise interprétation dans les environnements dynamiques **Coût en ressources :** exigences élevées en puissance de calcul et mémoire **Phase d'apprentissage :** temps initial nécessaire pour établir un profil comportemental fiable

- Kitsune (IDS léger basé sur un réseau neuronal pour IoT)
- Anomaly-based IDS pour réseaux ZigBee et MQTT

Ainsi, la détection comportementale est de plus en plus privilégiée dans les environnements IoT pour sa capacité à faire face aux menaces évolutives, malgré les contraintes de ressources et de précision qu'elle impose.

Limitations des solutions actuelles

Les solutions traditionnelles de détection d'intrusion (que ce soit par signature, comportementale ou hybride) présentent plusieurs limitations, notamment dans le contexte spécifique des réseaux IoT.

Limitations générales :

- **Taux élevé de faux positifs** : Les environnements IoT dynamiques compliquent la distinction entre comportements normaux et menaces.
- **Difficulté d'adaptation aux nouvelles menaces** : Les systèmes classiques nécessitent des mises à jour constantes ou un réapprentissage face à l'émergence de nouvelles attaques.
- **Consommation élevée de ressources** : Les dispositifs IoT étant souvent limités en ressources, ces solutions peuvent être trop exigeantes pour eux.

Défis spécifiques au contexte IoT :

- **Hétérogénéité des réseaux et des dispositifs** : La diversité des objets connectés complique l'établissement de modèles de détection universels.
- **Attaques zero-day** : Les systèmes traditionnels sont vulnérables face aux attaques inconnues ou modifiées.
- **Besoin de solutions intelligentes** : Les solutions doivent évoluer vers l'intégration d'IA et d'apprentissage automatique pour améliorer la détection et la réactivité.

Conclusion

Ce chapitre a présenté un aperçu de l'Internet des objets (IoT), en mettant en évidence ses caractéristiques, ses applications et les défis en matière de sécurité. Nous avons constaté que les systèmes IoT, souvent déployés dans des environnements sensibles, présentent des vulnérabilités accrues.

Nous avons aussi étudié les systèmes IDS classiques, qui, bien que courants, ont des limites en termes de détection de nouvelles menaces et de consommation de ressources. Ces défis soulignent la nécessité de solutions plus adaptatives.

Dans le prochain chapitre, nous explorerons les solutions basées sur le deep learning, qui offrent des perspectives intéressantes pour améliorer la sécurité des systèmes IoT. L'intégration de ces technologies représente une stratégie clé pour renforcer leur résilience face aux menaces émergentes.

Chapter 2

Détection d'intrusion basée sur l'intelligence artificielle

2.1 Introduction

Après avoir présenté un état de l'art sur les objets connectés et la détection d'intrusions dans les réseaux à l'aide des IDS classiques, nous disposons désormais des bases nécessaires pour explorer les trois approches proposées en réponse aux hypothèses formulées dans l'introduction.

Ce chapitre se concentre sur la détection d'intrusion dans le contexte des réseaux IoT, en mettant l'accent sur les approches d'intelligence artificielle.

Nous commençons par définir le Machine Learning, en exposant ses fondements et ses spécificités. Nous introduisons ensuite l'apprentissage profond (Deep Learning) ainsi que différentes architectures de réseaux neuronaux.

Nous analysons par la suite les travaux existants, ce qui nous permettra de mieux comprendre l'utilisation du Deep Learning pour la détection d'intrusions dans les réseaux IoT.

Cette base théorique nous permettra ainsi d'aborder de manière plus approfondie les solutions de détection d'intrusions appliquées aux environnements IoT que nous traiterons dans les chapitres suivants.

2.2 Détection basée sur l'intelligence artificielle

L'intelligence artificielle (IA) désigne l'ensemble des techniques et algorithmes permettant à une machine de réaliser des tâches qui, traditionnellement, requièrent l'intelligence humaine : apprentissage, raisonnement, perception et prise de décision. Historiquement, on distingue deux grandes approches :[5]

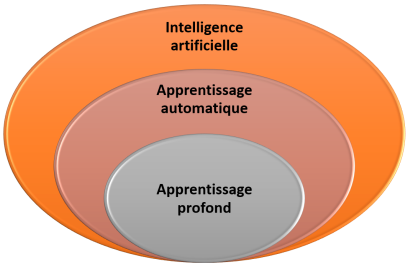


Figure 2.1: Domaines de l'intelligence artificielle

2.2.1 Machine Learning

Dans cette partie, nous allons d'abord définir le Machine Learning, puis présenter ses principaux types d'apprentissage.

Définition

L'apprentissage automatique, branche de l'intelligence artificielle, vise à créer des systèmes capables d'adapter et d'optimiser leurs performances en apprenant des données traitées. Plus précisément, il s'agit de développer des algorithmes capables :

- D'apprendre à partir des données, sans recourir à des règles codées manuellement.
- D'améliorer leurs performances au fil des exemples rencontrés (via des itérations d'entraînement).
- De réaliser diverses tâches telles que la classification, la régression ou le clustering, en s'appuyant sur des modèles statistiques. [6]

Types d'algorithmes d'apprentissage automatique pour la détection d'intrusion

[7]

Il existe divers types d'apprentissage automatique utilisés pour la détection d'intrusion, chacun ayant des caractéristiques spécifiques adaptées à différents scénarios. Cette section explore quelques-uns de ces algorithmes dans le contexte de la cybersécurité et de la détection d'attaques.

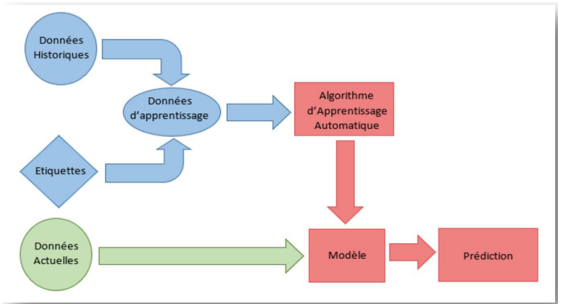


Figure 2.2: Enter Caption

Apprentissage supervisé L'apprentissage supervisé repose sur des ensembles de données étiquetées, où chaque échantillon est associé à un résultat attendu. Dans la détection d'intrusions, cette approche se traduit par la classification des échantillons en trafic normal ou attaque. Les modèles supervisés, entraînés sur ces données, permettent de prédire efficacement la classe de nouveaux échantillons.

Apprentissage non supervisé L'apprentissage non supervisé ne nécessite pas de données étiquetées. Utilisé pour le regroupement (clustering) ou la détection d'anomalies, il permet dans les IDS d'identifier automatiquement des comportements suspects en distinguant des groupes anormaux sans étiquettes prédéfinies.

Apprentissage semi-supervisé L'apprentissage semi-supervisé combine apprentissage supervisé et non supervisé. Seule une partie des données est étiquetée, tandis que l'autre est non étiquetée. Cette approche est particulièrement adaptée à la détection d'anomalies, permettant d'identifier des écarts par rapport au comportement normal du réseau.

Limites du Machine Learning classique

Bien que le Machine Learning ait démontré son efficacité, ses performances dépendent fortement de la qualité des données d'apprentissage et de l'étape critique d'ingénierie des caractéristiques (\textit{Feature Engineering}). Cette étape, qui nécessite l'intervention d'experts pour sélectionner manuellement les attributs pertinents, limite l'automatisation et l'adaptabilité des systèmes de détection.

Les avancées en apprentissage automatique ont permis de développer des modèles capables de classifier et de regrouper efficacement le trafic réseau à des fins de sécurité. Les premiers chercheurs ont testé des algorithmes simples, tels que k-Nearest Neighbor (KNN), la machine à vecteurs de support (SVM) et les cartes auto-organisatrices (SOM), obtenant de bons résultats sur des jeux de données comme KDD99, NSL-KDD, et DARPA.

Cependant, ces jeux de données sont aujourd'hui obsolètes. Ils contiennent non seulement des données normales, mais aussi des données d'attaques trop simplifiées, ce qui les rend inadaptés pour simuler l'environnement réseau complexe d'aujourd'hui. De plus, comme le montre notre étude, il est également difficile d'obtenir les résultats escomptés en analysant du trafic malveillant avec ces algorithmes sur des jeux de données plus récents

Pour surmonter ces limites, et grâce à la disponibilité croissante du Big Data, une nouvelle approche appelée **Deep Learning** (apprentissage profond) a émergé. Le Deep Learning permet aux modèles d'apprendre automatiquement des représentations abstraites à partir des données, sans nécessiter d'ingénierie manuelle des caractéristiques.

Dans la section suivante, nous introduirons donc l'apprentissage profond (Deep Learning), une approche plus avancée, capable de lever ces limitations et d'améliorer significativement la détection d'intrusions dans les environnements IoT.

2.2.2 Détection d'intrusion basée sur le deep learning

Dans cette partie, nous allons d'abord définir le deep Learning, puis présenter ses principaux types d'apprentissage.

définition

Le terme « deep learning » (apprentissage profond) désigne un type spécifique d'apprentissage automatique (machine learning), qui repose sur l'utilisation de réseaux de neurones (NN) à plusieurs couches pour résoudre des problèmes complexes. Ces réseaux « apprennent » à partir de vastes ensembles de données pour imiter, dans une certaine mesure, l'activité du cerveau humain, bien qu'ils soient encore loin d'en atteindre les capacités. Un réseau à une seule couche peut déjà fournir des approximations, mais l'ajout de couches cachées permet d'augmenter considérablement la précision. Un modèle d'apprentissage profond se compose généralement de plusieurs réseaux indépendants.

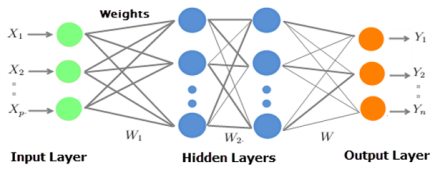


Figure 2.3: L'architecutre d'un modle de deep learning

Comparaison entre Machine Learning et Deep Learning

Table 2.1: Comparaison détaillée entre Machine Learning et Deep Learning

Caractéristiques	**Machine Learning**	**Deep Learning**
Structure	Algorithmes simples à moyennement complexes	Réseaux de neurones multicouches complexes
Volume de données	Fonctionne avec des ensembles de données moyens	Nécessite de grands ensembles de données
Temps d'entraînement	Relativement court	Long et intensif en calcul
Extraction des caractéristiques	Manuelle, nécessite une expertise domaine	Automatique, apprentissage des caractéristiques
Hardware requis	Ordinateur standard suffisant	Nécessite GPU puissant
Interprétabilité	Plus facile à interpréter	Fonctionne comme une "boîte noire"
Applications typiques	Classification simple, régression, clustering	Vision par ordinateur, NLP, reconnaissance vocale
Précision	Bonne pour des tâches simples	Excellente pour des tâches complexes
Flexibilité	Moins adaptable à de nouvelles données	Hautement adaptable
Coût de mise en œuvre	Relativement faible	Élevé (matériel + temps)

Quelques méthodes d'apprentissage profond

Cette section présente les principaux modèles d'apprentissage profond utilisés dans le domaine de la cybersécurité pour la détection d'intrusions. Comme illustré dans le tableau 1, dix approches majeures sont généralement employées. Parmi celles-ci figurent le réseau de neurones profond (DNN)„ le réseau de neurones récurrent (RNN) et le réseau de neurones convolutif (CNN), reconnu pour ses capacités d'extraction automatique de caractéristiques. D'autres techniques sont également utilisées, telles que le réseau de croyance profond (DBN), l'autoencodeur profond, ainsi que des méthodes spécialisées comme le transfert learning, l'apprentissage autodidacte ou encore le réseau neuronal réplicateur. Ces modèles permettent de concevoir des solutions capables d'identifier des attaques sophistiquées de manière plus autonome et efficace.

Deep Neural Network (DNN) Les **Deep Neural Networks** (DNN) sont des réseaux de neurones organisés en plusieurs couches successives, appelés aussi **Multilayer Perceptrons** (MLP). Ils se distinguent des réseaux neuronaux traditionnels (**Artificial Neural Networks**, ANN) par leur profondeur : ils comportent plusieurs couches cachées et un grand nombre de neurones.

Lorsqu'un réseau possède au moins deux couches cachées, il est considéré comme un réseau neuronal profond. Les DNN permettent de modéliser des données complexes en appliquant une série de transformations non linéaires.

Le concept de base, le **perceptron**, a été introduit par **Rosenblatt** en 1958. Il s'agit d'un modèle qui reçoit plusieurs entrées et génère une sortie après avoir appliqué une fonction d'activation non linéaire.

Un réseau DNN classique est constitué :

- d'une couche d'entrée qui reçoit les données,
- d'une ou plusieurs couches cachées qui effectuent des traitements,
- et d'une couche de sortie qui fournit le résultat.

L'information circule d'une couche à l'autre, en suivant un chemin précis. Les DNN sont principalement utilisés dans les tâches d'apprentissage supervisé, où l'objectif est d'ajuster les paramètres internes (poids et biais) pour obtenir les meilleures performances possibles.[8]

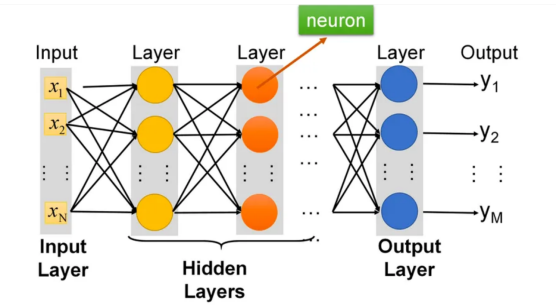


Figure 2.4: architecture de modele DNN

CNN

Le CNN est un type de réseau de neurones artificiels (ANN), inspiré par l'étude des cellules du cortex visuel. Sa caractéristique principale est de réduire le nombre de paramètres dans le réseau grâce à une connexion locale et des poids partagés, afin d'obtenir un certain degré d'invariance affine.[9]

Lors de l'utilisation du modèle de réseau neuronal pour les tâches de détection, la structure du réseau a une grande influence sur les résultats de détection. Cependant, la plupart des dispositifs périphériques sont limités par la mémoire et les ressources de calcul , et ils ne peuvent pas stocker ni exécuter des modèles CNN complexes. Par conséquent, lors de la définition de la structure hiérarchique du modèle CNN, nous commençons principalement par ajuster les paramètres et optimiser la structure.

Tout en assurant la précision, la structure du modèle CNN reste relativement simple, comme le montre la Figure 2.5.

Le modèle CNN est composé de plusieurs couches essentielles : la couche de convolution, la couche de regroupement (pooling) et la couche entièrement connectée. La normalisation par lot (batch normalization) est appliquée après la convolution pour normaliser les données et éviter la disparition des gradients. La fonction d'activation ReLU est utilisée pour accélérer la convergence, remplaçant ainsi les fonctions traditionnelles comme Sigmoid ou tanh. La couche de regroupement (max-pooling) réduit la taille des sorties et le coût de calcul. Les couches entièrement connectées suivent, avec un nombre de neurones précisé dans la Figure 2.5.La dernière couche utilise Softmax pour la classification, attribuant la classe prédite en fonction du vecteur fractionnaire généré. [10]

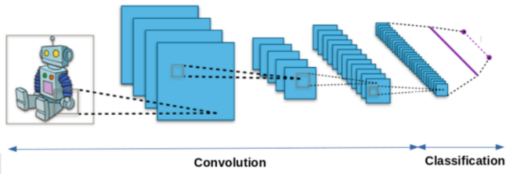


Figure 2.5: structure d'un CNN

RNN

Grâce à sa boucle de rétroaction, le **Recurrent Neural Network** (RNN)fonctionne comme une cellule mémoire : il peut reconnaître des motifs dans des séries temporelles et prédire des évolutions futures. Par exemple, dans le domaine automobile, les RNN peuvent analyser des trajectoires et anticiper des risques d'accidents.

De manière générale, les RNN sont capables de traiter des séquences d'entrée de taille variable et de produire des séquences de sortie également variables. Ils sont principalement utilisés pour modéliser des séquences, notamment pour :

- Prédire une valeur future à partir d'une série de mesures passées,
- Générer une série de mots à partir d'une image (par exemple, pour le sous-titrage automatique),
- Traduire automatiquement une série de mots d'une langue vers une autre, même si la longueur des séquences diffère.

Cependant, les RNN classiques présentent certaines limites, notamment des difficultés d'entraînement et une capacité de mémorisation réduite.

Table 2.2: Avantages et Inconvénients des RNN

Avantages	**Inconvénients**
Capacité à modéliser des séquences et à capturer les dépendances temporelles.	Problème du gradient qui disparaît ou explose, limitant l'apprentissage sur de longues séquences.
Flexibilité concernant la taille des entrées.	Entraînement lent et difficilement parallélisable.
Mémoire à court terme utile pour des applications comme le traitement de texte, de voix ou de données IoT.	Sensibilité élevée à l'ordre des données.

Le tableau 2.2 présente une analyse détaillée de leurs forces et faiblesses. [11]

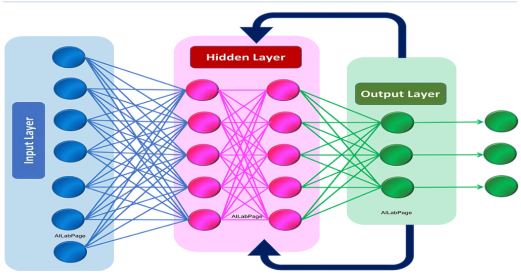


Figure 2.6: architecture de modele RNN

Ouverture vers le LSTM Pour surmonter ces faiblesses, notamment la difficulté à apprendre des dépendances à long terme, une architecture améliorée a été développée : le Long Short-Term Memory (LSTM). Grâce à ses mécanismes de portes (oubli, mise à jour, sortie), le LSTM offre une meilleure gestion de la mémoire et permet d'analyser des séquences longues avec plus de stabilité, ce qui le rend particulièrement adapté aux systèmes de détection d'intrusion dans les réseaux IoT.

LSTM

Plusieurs études ont démontré l'efficacité des réseaux LSTM (Long Short-Term Memory) pour la classification, le traitement et la prédiction de séries temporelles de longue durée [30–32]. Ces réseaux utilisent une mémoire interne qui leur permet de conserver les informations des étapes précédentes, un aspect essentiel dans le

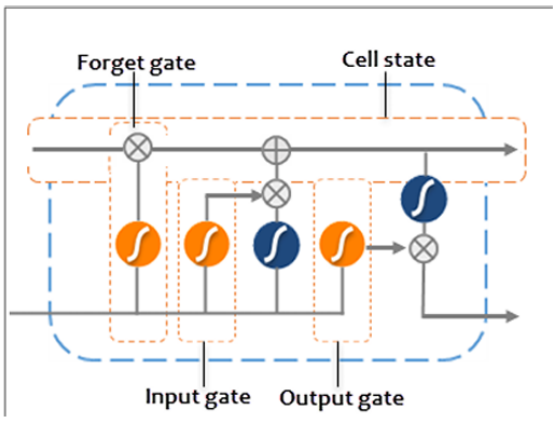


Figure 2.7: cellule LSTM

processus d'apprentissage. Ils sont également capables de gérer des représentations bruitées et des valeurs continues. Toutefois, les LSTM présentent un risque élevé de surapprentissage (overfitting), et l'application de la méthode de régularisation par "dropout" reste complexe à mettre en œuvre efficacement.

Le LSTM est un type spécifique de réseau de neurones récurrents (RNN), conçu pour résoudre les problèmes de disparition ou d'explosion du gradient lors de l'entraînement sur de longues séquences. Contrairement aux RNN standards qui utilisent uniquement une couche tanh, les LSTM améliorent les capacités de prédiction temporelle grâce à des portes mémoire spécifiques permettant d'oublier ou de conserver certaines informations.

2.3 Analyse des Méthodes de Détection d'Attaques

Dans cette section, nous examinerons les travaux antérieurs concernant la détection des attaques dans le domaine de l'Internet des objets (IoT), ainsi que les recherches dédiées aux systèmes de détection d'intrusions (IDS).

Samariya et al. (2023) ont mené une étude approfondie sur les techniques d'apprentissage profond pour la détection d'anomalies, mettant en évidence l'importance de jeux de données récents et fiables pour garantir une détection précise des intrusions. Leurs travaux ont notamment souligné que des ensembles de données tels que KDD99 et NSL-KDD, utilisés historiquement, ne reflètent plus efficacement les cybermenaces actuelles. [12]

Yin et al. (2021) ont proposé un modèle hybride pour la détection d'anomalies dans des séries temporelles de l'IoT, combinant un autoencodeur récurrent, un CNN et un DNN. Leur modèle a atteint une précision de 99,6 [13]

Wang et al. (2021) ont développé un réseau hiérarchique profond pour l'analyse du trafic malveillant, utilisant des CNN et des unités GRU pour traiter les données brutes au niveau des paquets. Bien que leur modèle ait montré de bonnes performances pour certaines attaques, ses performances étaient moins efficaces dans d'autres scénarios.[**?**]

Hassan et al. (2020) ont proposé une approche hybride en combinant CNN avec un modèle LSTM régularisé par "Weight Dropping" (WDLSTM). Cette méthode a atteint une précision de 97,1 % en classification binaire et de 98,4 % en classification multi-classes sur le jeu de données UNSW-NB15, et a permis de réduire le surapprentissage. [14]

Conclusion

Dans ce chapitre, nous avons commencé par définir ce qu'est l'apprentissage profond (deep learning), en présentant ses fondements et ses spécificités. Nous avons ensuite introduit différentes architectures de réseaux neuronaux, telles que le MLP, le RNN, le CNN et les autoencodeurs, ainsi que le processus d'entraînement des réseaux et les défis associés, notamment liés à l'optimisation. nous avons analysé des travaux existants, ce qui nous permet de mieux comprendre l'utilisation du deep learning pour la détection d'intrusions dans les réseaux IoT. Cette base théorique nous permettra ainsi de mieux aborder les solutions de détection d'intrusions appliquées aux environnements IoT que nous traiterons dans les chapitres suivants.

Table 2.3: Enquête sur les Attaques et les Approches de Détection Appliquées par les Chercheurs

Année	**Type de Vulnérabilité**	**Méthode de Détection**	**Caractéristiques**	**Jeu de Données**	**Précision**	**Réf**
2020	Web Traffic	Random Forest, Gradient Boosting Machine	Généralisabilité	CSIC-2010v2, CICIDS2017, NSL-KDD, UNSW NB15	94,46%	[4]
2020	Attaques Hôte ou Réseau	Hybride Decision Tree et KNN	Sécurité Réseau	NSLKDD, KDD-Cup99	99,33%	[5]
2019	Ping of Death, Denial of Service, Host Attack	EM, SVM, Naïve Bayes, KNN, HNB	Améliore la robustesse des méthodes contemporaines	NSL-KDD, KDD-Cup99, ADFA-LA, AFDA-WD	78%	[6]
2019	Attaques U2R et R2L	FAIS, FCAAIS	Détection d'attaques nouvelles	NSL-KDD	88%	[7]
2018	DoS, DDoS	ANN, K-Means, SVM	Catégorisation des attaques réseau	KDD-99, NSL-KDD	97,25%	[8]
2020	Attaque niveau Paquet	LSTM, OHE	Extraction de caractéristiques au niveau paquet	ISCX2012, USTC-TFC2016, CICIDS2017	99,74%	[9]
2019	U2R, R2L, DoS	LSTM avec LR, NB, KNN, SVM	Performance améliorée	KDD-Cup99, NSL-KDD, UNSW NB-15, WSN-DS, CICIDS2017	93,50%	[10]
2021	Trafic Réseau	Approche Ensemble avec C4.5, Random Forest	Haute Précision et Efficacité	NSL-KDD, AWID, CIC-IDS2017	99,52%	[11]
2018	DoS, DDoS	NB, Decision Table, SGD	Haute Précision	NSL-KDD	99,93%	[12]
2018	Trafic Réseau	SVM, Random Forest	Amélioration Performance	NSL-KDD	99,33%	[12]

Chapter 3

Méthodologie et Conception

Introduction

Après avoir présenté toute la théorie nécessaire au développement de notre simulateur, nous passons maintenant à la deuxième partie pour exposer notre travail. Dans ce chapitre, nous présentons notre contribution ainsi que les différents outils que nous avons utilisés. Tout d'abord, nous avons choisi un simulateur pour modéliser des attaques dans un environnement IoT . Le simulateur choisi permet la création de scénarios d'attaque réalistes. Par la suite, nous détaillons le jeu de données utilisé dans notre recherche. Enfin, nous présentons notre modèle.

3.1 Objectifs et Démarche générale

Cette section présente les objectifs de notre méthodologie ainsi que la démarche générale adoptée pour la conception de notre solution intelligente de détection d'intrusion.

3.1.1 Objectifs de la méthodologie

Ce chapitre présente la méthodologie adoptée pour la mise en œuvre de notre solution intelligente. Il couvre les étapes clés allant du choix des outils de simulation et des données, au développement et à l'évaluation des modèles d'intelligence artificielle. L'objectif est de concevoir une architecture efficace et reproductible, capable de traiter des données IoT, les analyser, et détecter des comportements anormaux de manière intelligente.

3.1.2 Démarche générale adoptée

Dans le cadre de ce projet, nous proposons un modèle intelligent de détection d'intrusion basé sur le deep learning, destiné à surveiller les applications IoT et à identifier automatiquement les comportements anormaux. L'objectif est de renforcer la sécurité des réseaux IoT en garantissant la disponibilité, l'intégrité et la confidentialité des dispositifs connectés. Notre approche se divise en deux grandes phases complémentaires : 1. Simulation d'un environnement IoT

Nous commençons par la mise en place d'un environnement de simulation reproduisant un réseau IoT. Ce dernier génère diverses communications entre objets connectés . Cette étape permet de simuler à la fois des comportements légitimes et des activités malveillantes, couvrant ainsi un large éventail de scénarios réalistes.

2. Détection d'intrusion par intelligence artificielle

La deuxième phase consiste à intégrer un modèle de deep learning capable de classer les données issues de la simulation entre comportements normaux et comportements suspects. Ce modèle vise à assurer une détection automatique

et rapide des intrusions, indépendamment du point du réseau concerné, facilitant ainsi la réaction aux menaces

3.1.3 Méthodologie proposée

La méthode proposée repose sur l'utilisation d'un réseau de neurones pour analyser le jeu de données Bot-iot, qui contient des vecteurs d'attaques modernes. Dans ce travail, nous avons développé une approche permettant d'identifier le trafic réseau anormal à l'aide de techniques d'apprentissage profond, n combinant un modèle hybride basé sur **CNN** (Convolutional Neural Network) pour extraire les caractéristiques spatiales et **LSTM** (Long Short-Term Memory) pour capturer les dépendances temporelles.

Analyse des simulateurs existants

3.4.1 Évaluation des outils de simulation disponibles

Il existe de nombreux outils dédiés à la simulation, appelés simulateurs réseau, parmi lesquels on peut citer :

- **NS2 (Network Simulator 2)** : un simulateur à événements discrets permettant d'exécuter divers scénarios sur des topologies définies par l'utilisateur. Il est accompagné de NAM (Network Animator) pour la visualisation graphique des résultats .
- **NS3 (Network Simulator 3)** : successeur de NS2, c'est un simulateur à événements discrets principalement utilisé dans la recherche et l'enseignement. Il est distribué en open source sous licence GNU GPLv2 .

- **NeSSi (Network Security Simulator)** : un simulateur conçu pour les réseaux, intégrant des fonctionnalités spécifiques à la sécurité informatique .
- **OMNeT++** : un framework de simulation modulaire basé sur C++, particulièrement adapté à la conception de simulateurs réseau .
- **COOJA** : fourni avec le système d'exploitation Contiki, ce simulateur est spécialement conçu pour les réseaux de capteurs. Il permet de tester les communications entre nœuds et d'interagir avec les capteurs, offrant ainsi un environnement de test économique pour les développeurs.
- **Boson NetSim Network Simulator** : un outil de simulation du matériel et des logiciels Cisco, destiné à l'apprentissage des commandes Cisco IOS .

Justification du simulateur choisi

NS-3 est largement reconnu comme un excellent outil open source de simulation réseau, utilisé dans la recherche et l'enseignement en réseau. Ces derniers temps, l'intégration des algorithmes d'intelligence artificielle dans la recherche réseau suscite un intérêt croissant, de nombreux chercheurs privilégiant des frameworks open source tels que TensorFlow et PyTorch. Toutefois, l'intégration de ces frameworks de machine learning au niveau du code source des outils de simulation s'avère complexe en raison de leur développement indépendant. Il est donc plus pratique et plus simple d'établir une connexion entre les deux à travers une transmission de données entre processus.

NS-3 est l'un des principaux simulateurs de réseau open source au niveau des paquets, utilisé pour la recherche en réseau. Il dispose d'un noyau hautement performant, capable de supporter la parallélisation en cluster pour de grands scénarios, d'exécuter du code réel et d'interagir avec des bancs d'essai (testbeds).

Différences clés par rapport à d'autres simulateurs réseau :

- **Orientation ligne de commande Unix** : contrairement aux simulateurs qui proposent un environnement de développement intégré (IDE).
- **Développement des simulations en C++ et Python** : à la différence des outils utilisant un langage de simulation spécifique au domaine.

3.1.4 Préparation des données

Sélection du jeu de données

Dans le cadre de cette recherche, nous avons exploité le jeu de données BoT-IoT, élaboré par le Centre de recherche en cybersécurité de l'Université UNSW Canberra.

Ce dataset est très prisé au sein de la communauté scientifique pour évaluer les systèmes de détection d'intrusion dans des environnements IoT.

Le jeu de données BoT-IoT englobe une variété étendue de scénarios d'attaques, incluant des attaques DDoS, DoS, des scans de ports, des infiltrations, ainsi que diverses activités associées aux botnets. Sa particularité réside dans la création d'un trafic réaliste, mélangeant des flux légitimes et malveillants dans un environnement IoT simulé, comprenant des dispositifs tels que des capteurs et des caméras intelligentes.

Un des principaux atouts de BoT-IoT est la richesse de ses caractéristiques : chaque flux est associé à plus de 30 attributs, incluant des informations telles que l'horodatage, l'adresse IP, le port, la taille des paquets, ainsi que des mesures spécifiques du trafic réseau (durée, débit, etc.). Cette granularité est cruciale pour entraîner efficacement les modèles d'apprentissage automatique, leur permettant de mieux différencier les comportements normaux des activités suspectes.[15]

Sous-ensemble de 10% du jeu de données BoT-IoT

Le jeu de données BoT-IoT initial comprend environ 73 millions d'enregistrements. Afin de faciliter son traitement et d'améliorer l'équilibre des classes, un sous-ensemble représentant 10% du volume total a été généré, aboutissant à un ensemble d'environ 6 millions d'enregistrements. La création de ce sous-ensemble a consisté à inclure l'intégralité du trafic normal ainsi que les attaques de type Reconnaissance et Vol d'Information.[16] Compte tenu de la prédominance des attaques DoS et DDoS dans le jeu de données original (environ 30 millions d'enregistrements), une sélection restreinte de deux millions d'enregistrements pour ces deux types d'attaques a été opérée, cette démarche a permis d'obtenir une meilleure répartition entre trafic normal et attaques, ainsi qu'une distribution plus homogène entre les différentes catégories d'attaques. La répartition des échantillons par type est présentée dans le tableau suivant

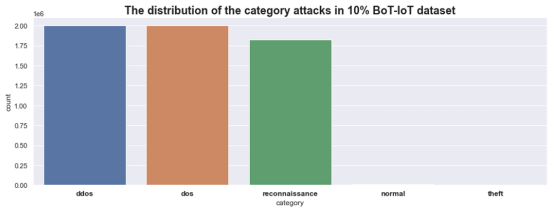


Figure 3.1: distrubition du dataset

Table 3.1: Répartition des types de trafic dans le sous-ensemble BoT-IoT (10 %)

Catégorie	**Nombre d'échantillons**
DDoS	2,000,000
DoS	2,000,000
Reconnaissance	1,821,639
Normal	9,543
Vol d'information (Theft)	1,587

Prétraitement du jeu de données Bot-IoTs

Cette étape consiste à transformer l'ensemble de données brut pour le rendre exploitable par les méthodes d'apprentissage profond. Elle comprend trois sous-étapes principales :

Standardisation : elle permet de mettre toutes les données sur la même échelle, avec des valeurs réparties entre 0 et 1 selon une distribution normale standard, garantissant ainsi l'homogénéité du traitement.

Normalisation : cette phase transforme les données pour éviter les valeurs négatives, qui peuvent perturber l'apprentissage des réseaux de neurones. L'ensemble des données est ainsi ramené entre 0 et 1.

Data Cleaning (Nettoyage des données) Les données sont nettoyées en éliminant les valeurs manquantes ou incorrectes pour assurer la qualité du jeu de données.

Sélection des caractéristiques

Dans cette étape, les meilleures caractéristiques (features) sont choisies pour le modèle. Cette phase est cruciale en apprentissage profond, car elle influence directement les performances du modèle. L'utilisation de caractéristiques inadaptées peut entraîner de mauvais résultats. Ainsi, nous avons sélectionné les attributs les plus pertinents à utiliser. Pour le jeu de données Bot-IoT, nous avons retenu quatre caractéristiques : « dur », « rate », « srate » et « drate ». Ces paramètres représentent des aspects liés au temps et à la durée, qui jouent un rôle essentiel dans la classification des attaques

Type de caractéristique	**Description**
Proto (Nominal)	Représentation textuelle des protocoles de transaction présents dans le flux réseau
Saddr (Nominal)	Adresse IP source
Sport (Entier)	Numéro de port source
Daddr (Nominal)	Adresse IP de destination
Dport (Entier)	Numéro de port de destination
Spkts (Flottant)	Nombre de paquets envoyés de la source vers la destination
Dpkts (Flottant)	Nombre de paquets envoyés de la destination vers la source
Sbytes (Flottant)	Nombre d'octets envoyés de la source vers la destination
Dbytes (Flottant)	Nombre d'octets envoyés de la destination vers la source
State (Nominal)	État de la transaction
Stime (Horodatage)	Heure de début de l'enregistrement
Ltime (Horodatage)	Heure de fin de l'enregistrement
Dur (Flottant)	Durée totale de l'enregistrement
Attack (Binaire)	Étiquette de classe : 0 pour trafic normal, 1 pour trafic d'attaque

Table 3.2: Description des caractéristiques sélectionnées

3.1.5 Choix de l'algorithme d'apprentissage

Nous avons adopté une approche hybride CNN-LSTM pour améliorer la détection des attaques. Le modèle commence par une couche d'entrée recevant 45 caractéristiques du trafic réseau. Il intègre **3 couches CNN** pour extraire les motifs importants, suivies de **2 couches LSTM** pour capturer les dépendances temporelles. Enfin, une couche de sortie avec activation *softmax* assure la classification binaire entre trafic normal et attaque.

Cependant, avant de concevoir ce modèle hybride, il a été jugé pertinent d'évaluer d'abord la performance d'un modèle **CNN seul** sur le jeu de données pré-traité. Cette étape intermédiaire a permis de mieux comprendre la capacité du CNN à capturer les motifs pertinents pour la détection d'intrusions, et de disposer d'une base de comparaison pour le futur modèle hybride.

Ainsi, la méthodologie suivie s'est déroulée en deux étapes principales :

- **Premièrement**, conception, entraînement et évaluation d'un modèle CNN classique.
- **Deuxièmement**, développement ultérieur d'un modèle hybride **CNN-LSTM** pour analyser l'apport du module LSTM sur les performances globales du système de détection.

Division des données (Data Splitting)

la division des données est une étape essentielle. Elle consiste à répartir le jeu de données en plusieurs sous-ensembles afin de permettre à l'algorithme d'apprendre, de valider et d'évaluer ses performances sur des données qu'il n'a jamais vues. Cette démarche permet d'éviter le surapprentissage (overfitting), où le modèle apprend uniquement par cœur les données d'entraînement sans réellement généraliser sur de nouvelles situations.

Dans le cadre de ce projet, le jeu de données bot-iot a été divisé selon les proportions suivantes :

75 des données pour l'entraînement du modèle (training),

25 pour le test, afin d'évaluer les performances,

Cette division a été appliquée à l'ensemble des modèles individuels (standalone), qu'ils soient issus de l'apprentissage automatique ou profond.

Training (Entraînement)

Les modèles sont entraînés avec le jeu de données prétraité. Les techniques d'optimisation comme l'early stopping, la régularisation L2 et le dropout sont utilisées pour éviter le sur-apprentissage.

Testing (Test)

Les modèles sont testés pour évaluer leurs performances

Une fois le modèle entraîné, il est testé sur un jeu de données de test pour vérifier sa capacité à détecter les intrusions et autres comportements malveillants[17]

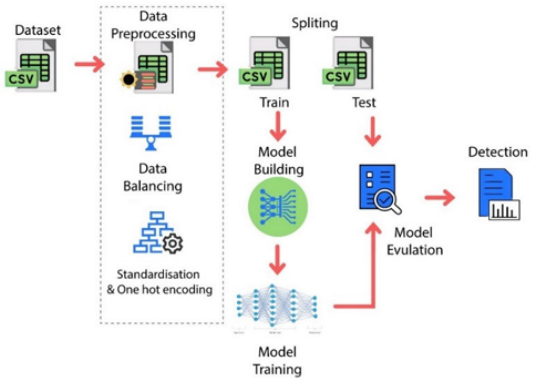


Figure 3.2: Approche utilisée pour détecter les attaques

3.1.6 Métriques et évaluation

Dans cette section, nous analyserons les performances des modèles IDS, en nous concentrant sur trois critères principaux : l'exactitude (Accuracy), la précision (Precision) et le taux de faux positifs (False Positive Rate).

Validation et évaluation Tous les résultats obtenus ont été évalués à l'aide de plusieurs métriques afin d'en vérifier l'efficacité. Les métriques utilisées sont : l'Accuracy (précision globale), la Loss (fonction de perte), le F1-score, la Précision et le Recall (rappel).

Ces indicateurs s'appuient sur la matrice de confusion, une table 2×2 qui recense les quatre issues possibles d'un classifieur binaire :

- **True Positive (TP)** : nombre de cas positifs correctement classés,
- **True Negative (TN)** : nombre de cas négatifs correctement classés,
- **False Positive (FP)** : nombre de cas négatifs incorrectement classés comme positifs,
- **False Negative (FN)** : nombre de cas positifs incorrectement classés comme négatifs.

Définitions des métriques L'**Accuracy** est définie par l'équation suivante, comme le rapport entre le nombre de bonnes prédictions (TP + TN) et le nombre total d'échantillons :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.1.1}$$

La **Loss** utilisée est la *binary cross-entropy*, exprimée par :

$$\text{Loss} = -\left[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})\right] \tag{3.1.2}$$

où y est la valeur réelle et $\hat{y}$ l'estimation du modèle.

La **Précision** mesure le ratio d'éléments correctement classés parmi ceux prédits comme positifs :

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3.1.3}$$

Le **Recall** est le ratio d'éléments correctement classés parmi l'ensemble des éléments réellement positifs :

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3.1.4}$$

Le **False Positive Rate (FPR)** mesure le taux d'éléments négatifs incorrectement classés comme positifs :

$$\text{FPR} = \frac{FP}{TN + FP} \tag{3.1.5}$$

Enfin, le **F1-score** est la moyenne harmonique de la précision et du rappel :

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.1.6}$$

Conclusion

Dans ce chapitre, nous avons présenté notre démarche méthodologique, du choix du simulateur NS-3 à la préparation du jeu de données BoT-IoT. Nous avons ensuite détaillé notre approche basée sur un modèle hybride CNN-LSTM pour la détection intelligente d'intrusions dans les réseaux IoT. Cette méthodologie structurée constitue la base du développement et de l'évaluation de notre solution.

Chapter 4

Implementations et Résulats

Introduction

Dans ce chapitre, nous présentons l'environnement d'exécution utilisé pour développer et tester nos modèles d'apprentissage profond, ainsi que la configuration matérielle et logicielle mise en place. Nous décrivons ensuite les résultats expérimentaux obtenus à travers l'entraînement et l'évaluation de nos modèles CNN et LSTM-CNN sur des jeux de données spécialisés en détection d'attaques réseau. Enfin, nous abordons l'implémentation pratique de notre approche dans un environnement de simulation 5G, en détaillant les étapes de simulation, d'intégration du modèle et d'analyse des résultats.

4.1 Environnement d'exécution et Configuration du système

Dans cette partie, nous décrivons l'environnement d'exécution ainsi que la configuration du système utilisé pour développer et entraîner le modèle de réseau de neurones.

4.1.1 Environement d'execution

Dans cette partie, nous avons utilisé le langage Python en raison de sa riche bibliothèque d'outils dédiés à l'IA. Pour la création et l'entraînement du modèle de réseau de neurones, nous avons choisi TensorFlow et Keras, qui offrent des fonctionnalités avancées pour le développement de modèles profonds. Le prétraitement des données a été effectué avec Pandas et NumPy, permettant une manipulation

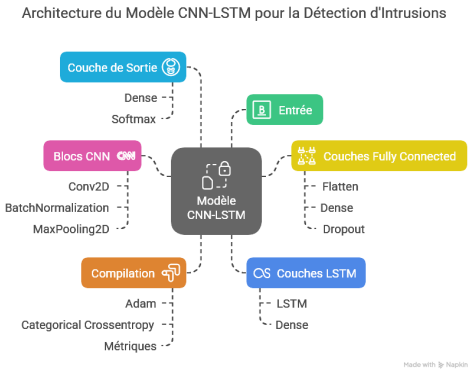


Figure 4.1: architecture de modele CNN-LSTM

efficace des données. Le tout a été développé dans un environnement interactif avec Google Colab pour des tests rapides .

4.1.2 Configuration du système

L'environnement expérimental a été réalisé sur un ordinateur portable Lenovo, équipé d'un processeur Intel® Core™ i5 et de 20 Go de RAM. Le modèle a été développé en utilisant les frameworks TensorFlow et Keras , qui offrent des bibliothèques puissantes et flexibles pour la mise en œuvre de solutions d'apprentissage profond.

4.2 Résultats expérimentaux

Afin d'évaluer les performances de notre approche, nous présentons ci-dessous l'architecture du modèle CNN-LSTM utilisé dans nos expérimentations:
[18]

4.2.1 Résultats du modèle CNN-LSTM

Cette sous-section présente les résultats de notre modèle proposé. Cette méthode repose sur une combinaison de CNN et LSTM pour la sécurité des réseaux. Les résultats ont été obtenus en divisant l'ensemble de données en 75 pour l'entraînement et 25 pour la validation. Lors de l'expérimentation, le modèle a été formé en utilisant le jeu de données Bot-iot, englobant à la fois le trafic bénin et malveillant. L'entraînement a été réalisée sur Google Colab, en exécutant 25 époques et en utilisant l'optimiseur Adam. Nous avons utilisé la bibliothèque Keras pour construire nos classificateurs, avec TensorFlow comme moteur d'exécution.

Rapport de classification

Le tableau 4.1présente une évaluation détaillée de la classification binaire de notre système en utilisant des métriques telles que la précision, le rappel, le score F1 et le support. Il est évident que la performance du modèle varie selon les différentes classes, notamment pour les deux catégories principales (trafic normal et attaques).

Concernant la classe **"Attaques"**, le modèle atteint des performances exceptionnelles avec une précision de **99,99%**, un rappel de **99,99%**, et un score F1 de **99,99%**. Ces résultats montrent que le modèle est extrêmement efficace pour identifier les attaques.

Pour la classe **"Trafic normal"**, bien que la précision soit légèrement plus basse (**99,25%**), le modèle conserve une bonne capacité à détecter le trafic normal avec un rappel de **99,6%** et un score F1 de **99,4%**. Cela montre que, bien qu'il y ait un léger compromis, le modèle reste performant. En ce qui concerne les moyennes globales, le modèle affiche une **précision** de **99,99%**, un **rappel** de **99,99%**, et un **score F1** de **99,99%**, ce qui reflète une excellente performance générale.

Table 4.1: Rapport de classification

Catégorie	**Précision**	**Rappel**	**Score F1**	**Support**
Trafic normal	99,25%	99,62%	99,43%	2.385
Attaques	99,99%	99,99%	99,99%	1.448.193
Précision globale	99,99%			
Moyenne macro	99,62%	99,81%	99,70%	1.450.578
Moyenne pondérée	99,99%	99,99%	99,99%	1.450.578

Graphiques d'accuracy et de perte : La Figure **??** présente l'évolution de l'accuracy et de la loss **??** pour l'entraînement et la validation, en fonction du nombre d'époques fixé à 13. Dans la Figure on observe que les performances du modèle s'améliorent au fur et à mesure de l'augmentation du nombre d'époques.

Confusion Matrix:

un tableau qui permet d'évaluer les performances d'un modèle de classification en affichant le nombre de prédictions correctes et incorrectes pour chaque classe (vraies positives, fausses positives, vraies négatives et fausses négatives)

4.3 Implémentation du Système

Dans cette partie, nous décrivons la mise en œuvre pratique de notre approche. Conformément à la méthodologie présentée précédemment, nous commençons

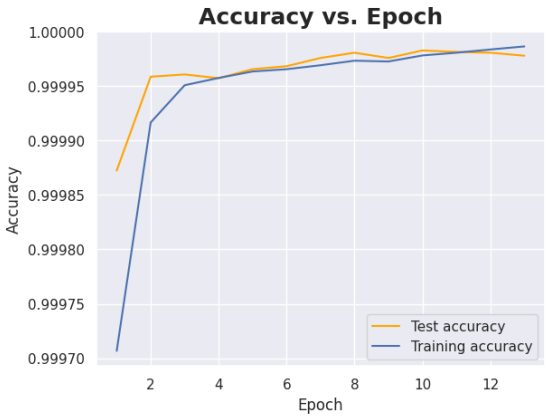


Figure 4.2: Accuracy vs appoch

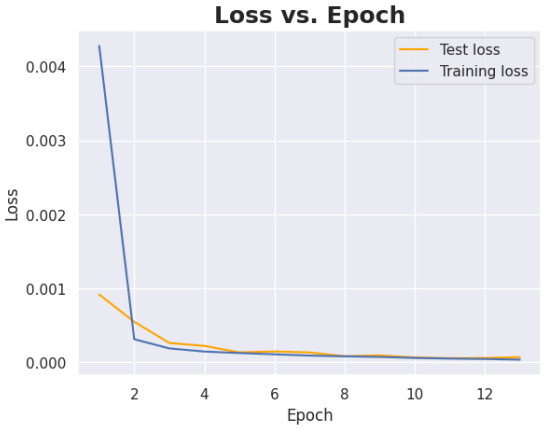


Figure 4.3: Loss vs Epoch

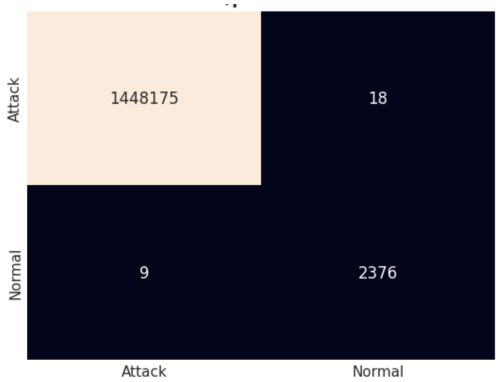


Figure 4.4: Confusion Matrix of the test/predicted attacks in Bot-iot using CNN

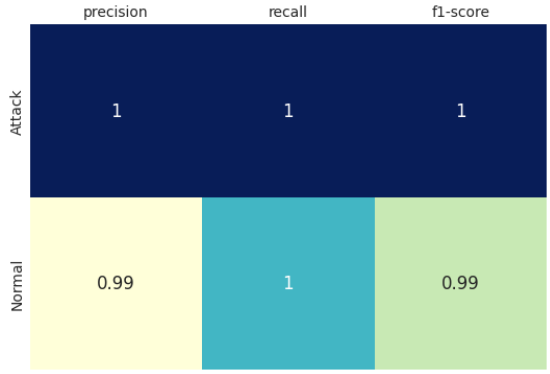


Figure 4.5: accracy,recall

par la simulation du trafic réseau, puis nous passons à l'intégration du modèle d'intelligence artificielle dans notre système.

4.3.1 Mise en place de l'environnement de simulation

Afin de simuler un réseau 5G, caractérisé par sa haute vitesse, sa faible latence et sa capacité à connecter un grand nombre d'appareils, et de tester les performances de détection d'attaques par l'IA, nous avons utilisé **ns-3**. Nous avons également intégré des scripts Python pour le traitement et l'analyse des données collectées à partir des simulations.

Scénario de simulation

Le scénario `5GNormalScenario` simule un réseau local filaire classique basé sur la technologie **CSMA** (Carrier Sense Multiple Access). Ce réseau est composé de 12 nœuds interconnectés, chacun étant configuré avec la pile de protocoles Internet, et l'adresse IP est attribuée à l'aide du module `Ipv4AddressHelper`.



Figure 4.6: execution de fichier 5g-.cc

Objectifs de la Simulation

L'objectif principal de ce scénario est d'évaluer et d'analyser les performances du réseau local en termes de **flux UDP**. Pour ce faire, plusieurs outils sont

utilisés :

- **Évaluation des performances du réseau :** L'accent est mis sur l'analyse des flux UDP entre les clients et le serveur.
- **Visualisation de la topologie :** Grâce à **NetAnim**, la topologie et les échanges réseau sont représentés graphiquement.
- **Capture de paquets :** Les paquets échangés sont capturés à l'aide de **Wireshark** via des fichiers **PCAP** générés par la simulation.
- **Génération de statistiques de performance :** **FlowMonitor** est utilisé pour collecter des données sur les flux, telles que le débit, la perte de paquets et le délai.

Configuration Réseau

La configuration du réseau pour ce scénario est la suivante :

- **Bande passante du lien CSMA :** Le réseau utilise un lien avec une bande passante de **1 Gbps**.
- **Délai de transmission :** Le délai de transmission entre les nœuds est de **6560 nanosecondes**.
- **Simulation d'un serveur UDP Echo :** Le serveur UDP Echo est configuré sur le nœud 0, et il reçoit des requêtes des clients.
- **Clients UDP :** Chaque autre nœud (nœuds 1 à 11) agit comme un client qui envoie **3 paquets de 1024 octets** toutes les **2 secondes** au serveur.

Analyse des Résultats

Les résultats de la simulation sont analysés avec différents outils de NS-3.

Wireshark Les fichiers **PCAP** générés par la simulation permettent une inspection détaillée des paquets échangés. Ces fichiers sont générés en activant la capture de paquets via la commande suivante dans le code NS-3 :

```
csma.EnablePcapAll("5g-normal-scenario");
```

Cela permet d'analyser le trafic à l'aide de **Wireshark** pour examiner les paquets envoyés entre les nœuds et leur impact sur le réseau.

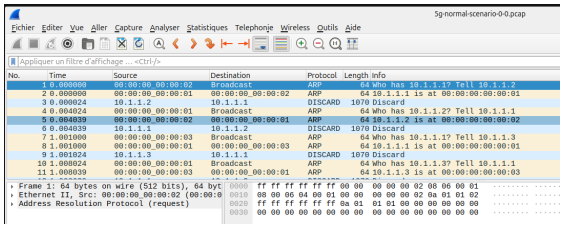


Figure 4.7: Analyse du Premier Paquet avec Wireshark

NetAnim NetAnim génère un fichier **.xml** qui est utilisé pour créer une animation de la topologie du réseau et des échanges de trafic. Ce fichier permet de visualiser la simulation en temps réel et d'étudier le comportement du réseau lors des échanges UDP.

Le fichier généré s'appelle `5g-normal-scenario.xml` et peut être ouvert dans l'outil **NetAnim** pour visualisation. **NetAnim**. La grille est organisée de la manière suivante :

Les nœuds sont disposés sur une grille avec un espacement de **20 unités** entre chaque nœud.

La disposition est organisée sur une **grille 4x3**.

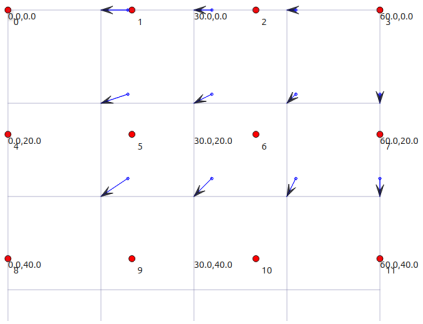


Figure 4.8: simulation sur NetAnim

2. Conversion des données pour l'IA Le script suivant extrait les logs générés par NS-3 et les transforme en un jeu de données exploitable par un modèle d'apprentissage automatique :

```
python ns3_to_ai.py
```

3. Détection d'attaques Enfin, ce script utilise un modèle d'IA pour analyser les données réseau et détecter d'éventuelles attaques :

```
python detect_attacks.py
```

Simulation d'attaques

- Introduction de trafic malveillant simulant des attaques de type DDoS.
- Génération d'un trafic excessif provenant de plusieurs UEs vers une même cible réseau.

DDoS (Distributed Denial of Service) Description : Inonder une cible (serveur, service, réseau) avec un trafic massif.

Simulation : Génération de trafic excessif depuis plusieurs UEs (User Equipments).

Mesures extraites : débit élevé, augmentation de la latence, perte de paquets, chute de disponibilité.

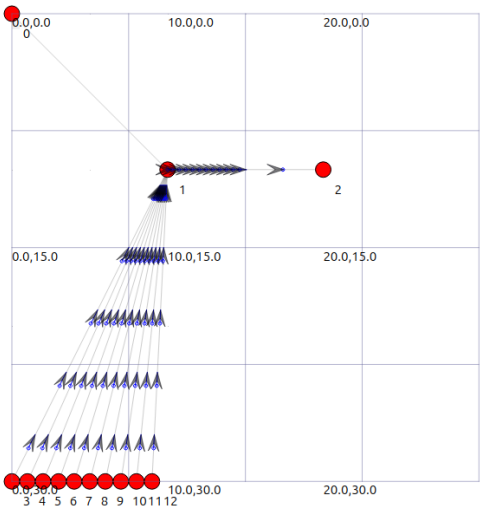


Figure 4.9: DDOS simulation

4.3.2 Résultats et observations

Malgré les différentes étapes de préparation et d'extraction des données, l'intégration directe du modèle d'intelligence artificielle (CNN-LSTM) dans l'environnement de simulation NS-3 n'a pas encore pu être finalisée. Cette étape s'est révélée complexe en raison des contraintes de compatibilité entre le simulateur réseau et les bibliothèques Python utilisées pour l'apprentissage .

Cependant, des efforts sont en cours pour réussir cette intégration d'ici la soutenance. À défaut d'une intégration directe, une démonstration vidéo montrant

le fonctionnement de la simulation sera réalisée afin d'illustrer le processus de détection d'attaques à partir des données générées par NS-3.

Conclusion

Dans ce chapitre, nous avons décrit en détail l'environnement de développement, la configuration matérielle et logicielle utilisée, ainsi que l'architecture de notre modèle CNN-LSTM conçu pour la détection d'intrusions dans les réseaux. L'évaluation du modèle sur le jeu de données BoT-IoT a permis d'obtenir des résultats très satisfaisants, avec des taux de précision, dépassant 99%. Ces performances témoignent de l'efficacité de notre approche hybride combinant les atouts des réseaux convolutifs pour l'extraction de caractéristiques et ceux des LSTM pour la capture des dépendances temporelles. Par ailleurs, l'implémentation dans un environnement de simulation 5G ouvre des perspectives prometteuses pour une intégration réelle dans des systèmes de cybersécurité intelligents et autonomes.

Conclusion

La conclusion de notre projet résume les résultats obtenus et ouvre des perspectives d'amélioration pour les travaux futurs. Nous abordons trois points principaux.

1. **Rappel du contexte et de la problématique :**

   L'Internet des Objets (IoT) a connu une adoption massive, facilitant la connectivité entre des milliards d'appareils. Cependant, cette interconnexion génère des vulnérabilités en matière de sécurité, notamment concernant la confidentialité et l'intégrité des données. Ce projet a été motivé par la nécessité d'améliorer la sécurité des environnements IoT, en particulier face aux menaces d'intrusion susceptibles de compromettre ces systèmes critiques.

2. **Résumé du rapport et des étapes clés du projet :**

   Au fil de notre étude, nous avons d'abord exploré les technologies IoT et les défis de sécurité associés. Par la suite, nous avons proposé une solution basée sur l'intelligence artificielle, et plus spécifiquement sur les algorithmes de deep learning, pour la détection d'intrusions. Les étapes clés incluent l'analyse des technologies IoT, la sélection des outils de simulation, la préparation des données, ainsi que la mise en œuvre des modèles d'IA. Les résultats montrent une nette amélioration dans la détection des comportements anormaux par rapport aux approches traditionnelles.

3. **Perspectives d'amélioration envisagées :**

   Plusieurs pistes d'amélioration peuvent être envisagées pour renforcer l'efficacité du Federated Learning dans les systèmes IoT. Tout d'abord, l'intégration de techniques de chiffrement avancé, telles que le chiffrement homomorphe ou les enclaves sécurisées, permettrait de garantir une meilleure confidentialité des échanges entre les dispositifs connectés et les serveurs. Ensuite, l'adoption de mécanismes d'apprentissage continu rendrait les modèles de détection plus réactifs face aux nouvelles formes d'attaques, en assurant une mise à jour régulière sans compromettre la confidentialité des données. Enfin, les recherches futures pourraient se focaliser sur l'évolutivité de ces

systèmes, afin de maintenir un haut niveau de sécurité même dans des environnements distribués à grande échelle, comportant un grand nombre de nœuds hétérogènes

En conclusion, ce projet a posé les bases d'une approche innovante pour la détection d'intrusions dans l'IoT, tout en ouvrant des perspectives pour de futures améliorations et recherches, afin de renforcer la résilience de ces systèmes face aux menaces émergentes.

Annexe

- Annexe 1 : Chargement et Prétraitement des Données
- Annexe 2: Définition de l'Architecture CNN-B
- Annexe 3 : Entraînement du Modèle avec Early Stopping
- Annexe 4 : Évaluation et Visualisation des Performances

.1 Chargement et Prétraitement des Données

```
# Chargement du jeu de donn es BoT-IoT
data = pd.read_csv(
    '/content/drive/MyDrive/BoT-IoT.csv',
    low_memory=False
)

# Affichage de la distribution des cat gories
data.category.value_counts()

# Filtrage des protocoles r seau
data = data.loc[
    (data['proto'] == 'tcp') |
    (data['proto'] == 'udp') |
    (data['proto'] == 'icmp') |
    (data['proto'] == 'arp') |
    (data['proto'] == 'igmp'),
    :
]

# Filtrage des  tats  de connexion
data = data.loc[
    (data['state'] == 'RST') |
    (data['state'] == 'REQ') |
    (data['state'] == 'INT') |
    (data['state'] == 'FIN') |
    (data['state'] == 'CON') |
    (data['state'] == 'ECO') |
    (data['state'] == 'ACC') |
    (data['state'] == 'PAR'),
    :
]
```

.2 Définition de l'Architecture CNN-B

```
# D finition de l'architecture CNN-B
input_shape = (24, 1, 1)
model = models.Sequential()

# Premier bloc de convolution
```

```
model.add(layers.Conv2D(
    filters=32,
    input_shape=input_shape,
    kernel_size=(3,3),
    activation='relu',
    padding='same'
))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(
    pool_size=(1, 1),
    padding='same'
))
```

.3 Entraînement du Modèle avec Early Stopping

```
# Configuration de l'entra nement du mod le
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_test, y_test),
    epochs=25,
    batch_size=2048,
    callbacks=callbacks
)
```

Bibliography

[1] Amazon Web Services. What is the internet-of-things (iot)? https://aws.amazon.com/what-is/iot/, 2022. Accessed: 2022-07-05.

[2] IoT Industriel. Architecture iot : L'essentiel à savoir, 2022. Consulté le 29 avril 2025. URL: https://iotindustriel.com/iot-iiot/architecture-iot-lessentiel-a-savoir/.

[3] C. Bidan, G. Hiet, L. Mé, B. Morin, and J. Zimmermann. Vers une détection d'intrusions à fiabilité et pertinence prouvables. *Revue de l'Électricité et de l'Électronique*, 9:75, 2006.

[4] F. Majorczyk. *Détection d'intrusions comportementale par diversification de COTS: application au cas des serveurs web*. Thèse de doctorat, 2008.

[5] Cégep de Jonquière. Qu'est-ce que l'intelligence artificielle? Consulté le 28 avril 2025. URL: https://vie-etudiante.cegepjonquiere.ca/qu-est-ce-que-l-intelligence-artificielle.html.

[6] C. Llorens, L. Levier, D. Valois, and B. Morin. *Tableaux de bord de la sécurité réseau*. Éditions Eyrolles, Paris, France, 2010.

[7] Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien. *Semi-Supervised Learning*. MIT Press, 2006. Consulté dans le cadre de l'étude des réseaux neuronaux.

[8] ResearchGate. A typical architecture of dnn. Consulté en avril 2025. URL: https://www.researchgate.net/publication/335845675/figure/fig2/[10]AS:804124836765697@1568729709531/A-typical-architecture-of-DNN.ppm.

[9] A. Waibel, T. Hanazawa, G. Hinton, K. Shikano, and K.J. Lang. Phoneme recognition using time-delay neural networks. *IEEE Transactions on Acoustics, Speech, and Language Processing*, 37(3):328–339, 1989. Étude sur la reconnaissance des phonèmes utilisant des réseaux neuronaux à délai temporel. doi:10.1109/29.21714.

[10] Benlahmar. Les réseaux de neurones convolutifs, 2018. Consulté le 29 avril 2025. URL: https://datasciencetoday.net/index.php/en-us/deep-learning/173-les-reseaux-de-neurones-convolutifs.

[11] Sudarshan S. Recurrent neural network (rnn). Consulté en avril 2025. URL: https://sudarshan-s.medium.com/recurrent-neural-network-rnn-20a619190586.

[12] H. Samariya et al. Toward deep learning based intrusion detection system: A survey. *ACM Computing Surveys*, 56(4):Article 71, 2023. doi:10.1145/3688574.3688578.

[13] Y. Yin et al. Hybrid model for time series anomaly detection in iot. *Journal of Computing and Security*, 45(2):113–125, 2021.

[14] R. Hassan et al. Hybrid cnn-lstm model for intrusion detection. *Journal of Network and Computer Applications*, 63:132–140, 2020.

[15] Kaggle. N-baiot dataset to detect iot botnet attacks. https://www.kaggle.com/datasets/mkashifn/nbaiot-dataset, 2020. [Online].

[16] BoTIoT Dataset. Botiot dataset, 2022. Consulté le 27 mars 2024. URL: https://research.unsw.edu.au/projects/bot-iot-dataset.

[17] Mohammed S. Alshehri and Jawad Ahmad. Skipgatenet: A lightweight cnn-lstm hybrid model with learnable skip connections for efficient botnet attack detection in iot. *IEEE Access*, PP(99):1–1, 2024. URL: https://www.researchgate.net/publication/378676597_SkipGateNet_A_Lightweight_CNN-LSTM_Hybrid_Model_with_Learnable_Skip_Connections_for_Efficient_Botnet_Attack_Detection_in_IoT, doi:10.1109/ACCESS.2024.3371992.

[18] M. A. Pynadath, K. J. Pavithra, S. E. Lobo, S. S. Murthy, and R. Bharathi. Anomaly detection and multi-output classification of iot attacks. In *Proceedings of the International Conference on Inventive Computation Technologies (ICICT)*, pages 1750–1757, April 2023.